



Faculty of Technology and Engineering

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

Date: / /

Laboratory Manual

Academic Year	:	2024-25	Semester	:	4
Course code	:	CSE206	Course name	:	DATABASE MANAGEMENT SYSTEM

Practical - 12

Aim: - Your task is to design and implement database functionalities for managing product information in an e-commerce system. The system should allow users to create, retrieve, update, and delete product data, compute discounts on products using a function, and maintain an audit trail for product updates using a trigger.

The screenshot displays the SQL Server Enterprise Manager interface with the following SQL queries executed:

```

CREATE TABLE Product (
    product_id INT PRIMARY KEY IDENTITY(1,1), -- Auto-Increment ID
    name VARCHAR(255) NOT NULL,
    category VARCHAR(255),
    price DECIMAL(10,2),
    stock INT
);

CREATE TABLE AuditTrail (
    audit_id INT PRIMARY KEY IDENTITY(1,1),
    product_id INT,
    old_name VARCHAR(255),
    old_category VARCHAR(255),
    old_price DECIMAL(10,2),
    old_stock INT,
    updated_at DATETIME DEFAULT GETDATE(),
    operation VARCHAR(50) -- 'UPDATE', 'DELETE', etc.
);

CREATE PROCEDURE InsertProduct
    @name VARCHAR(255),
    @category VARCHAR(255),
    @price DECIMAL(10,2),
    @stock INT

```

The Messages pane at the bottom of the SQL Server Enterprise Manager window shows the following output:

```

Commands completed successfully.
Completion time: 2025-02-27T10:04:06.7382649+05:30

```

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays the SQL script for the 'InsertProduct' stored procedure. The bottom pane shows the results of the execution, which is a table with one row.

```
-- Call the InsertProduct procedure
EXEC InsertP 'Laptop', 'Electronics', 50000, 10;
SELECT * FROM Product;
```

product_id	name	category	price	stock
1	Laptop	Electronics	50000.00	10

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays the SQL script for the 'GetProductDetails' stored procedure. The bottom pane shows the results of the execution, which is a table with one row.

```
-- Call the InsertProduct procedure
-- Call the GetProductDetails procedure
EXEC GetProductDetails 1; -- Assuming product_id = 1
SELECT * FROM Product;
```

product_id	name	category	price	stock
1	Laptop	Electronics	50000.00	10

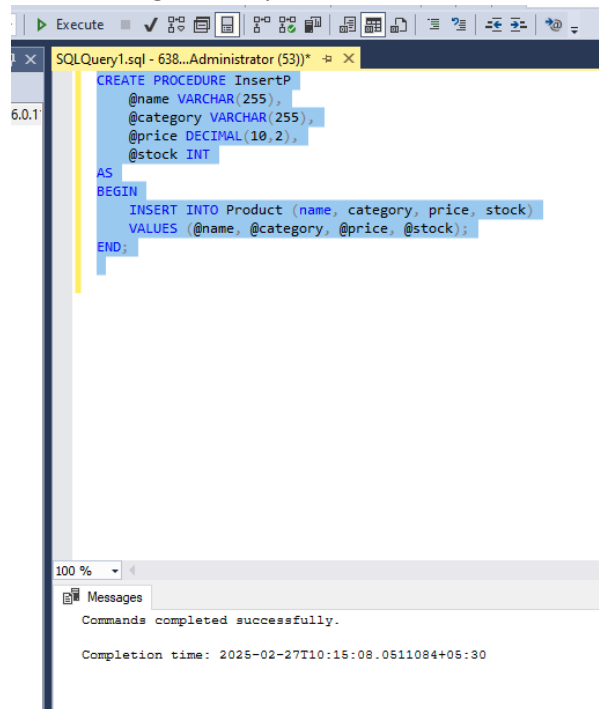
The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays the SQL script for the 'GetProductDetails' stored procedure. The bottom pane shows the results of the execution, which is a table with one row.

```
-- Call the InsertProduct procedure
-- Call the GetProductDetails procedure
EXEC GetProductDetails 1; -- Assuming product_id = 1
SELECT * FROM Product;
```

product_id	name	category	price	stock
1	Laptop	Electronics	50000.00	10

Task:

1. Create a stored procedure to insert a new product into the Product table.



The screenshot shows the SQL Server Enterprise Manager interface. The 'SQLQuery1.sql' file is open, displaying the following T-SQL code:

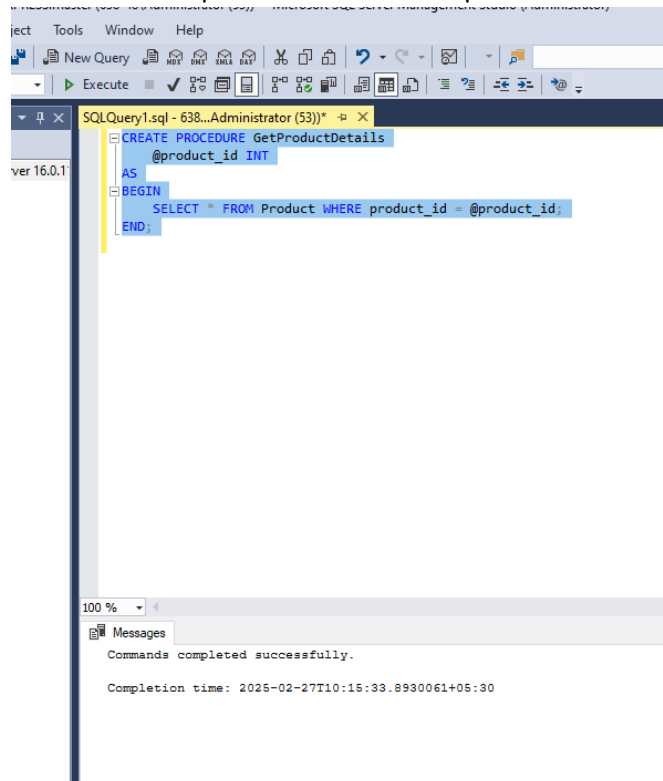
```
CREATE PROCEDURE InsertP
@name VARCHAR(255),
@category VARCHAR(255),
@price DECIMAL(10,2),
@stock INT
AS
BEGIN
INSERT INTO Product (name, category, price, stock)
VALUES (@name, @category, @price, @stock);
END;
```

The 'Messages' pane at the bottom shows the execution results:

```
Commands completed successfully.

Completion time: 2025-02-27T10:15:08.0511084+05:30
```

2. Create a stored procedure to retrieve product details by product_id.



The screenshot shows the SQL Server Enterprise Manager interface. The 'SQLQuery1.sql' file is open, displaying the following T-SQL code:

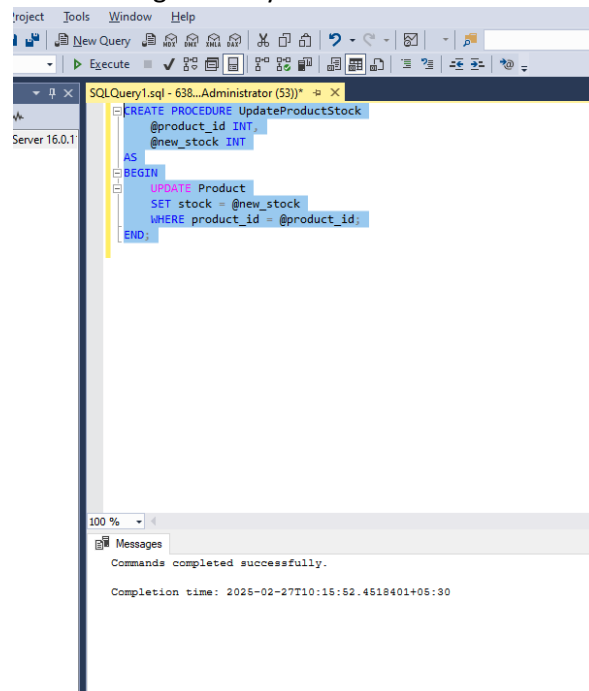
```
CREATE PROCEDURE GetProductDetails
@product_id INT
AS
BEGIN
SELECT * FROM Product WHERE product_id = @product_id;
END;
```

The 'Messages' pane at the bottom shows the execution results:

```
Commands completed successfully.

Completion time: 2025-02-27T10:15:33.8930061+05:30
```

3. Create a stored procedure to update the stock of a product.



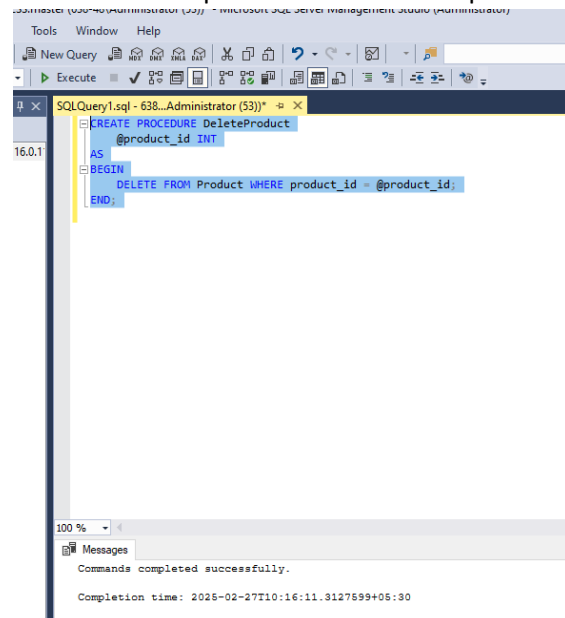
The screenshot shows the SQL Server Enterprise Manager interface. The 'SQLQuery1.sql - 638...Administrator (53)' window is open, displaying the following T-SQL code:

```
CREATE PROCEDURE UpdateProductStock
    @product_id INT,
    @new_stock INT
AS
BEGIN
    UPDATE Product
    SET stock = @new_stock
    WHERE product_id = @product_id;
END;
```

The 'Messages' pane at the bottom shows the execution results:

```
Commands completed successfully.
Completion time: 2025-02-27T10:15:52.4518401+05:30
```

4. Create a stored procedure to delete a product by product_id.



The screenshot shows the SQL Server Enterprise Manager interface. The 'SQLQuery1.sql - 638...Administrator (53)' window is open, displaying the following T-SQL code:

```
CREATE PROCEDURE DeleteProduct
    @product_id INT
AS
BEGIN
    DELETE FROM Product WHERE product_id = @product_id;
END;
```

The 'Messages' pane at the bottom shows the execution results:

```
Commands completed successfully.
Completion time: 2025-02-27T10:16:11.3127599+05:30
```

5. Create a function to calculate the discounted price of a product.

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a query window titled "SQLQuery1.sql - 638...Administrator (53))" containing the following T-SQL code:

```
CREATE FUNCTION CalculateDiscountedPrice
(
    @price DECIMAL(10,2),
    @discount DECIMAL(5,2) -- Discount percentage (e.g., 10 for 10%)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @price - (@price * @discount / 100);
END;
```

The bottom pane shows the Messages tab with the following output:

```
Commands completed successfully.

Completion time: 2025-02-27T10:16:28.4578012+05:30
```

6. Implement a trigger to maintain an audit trail of updates made to the Product table. Before any update, the trigger should insert a record into an AuditTrail table, storing the old product details along with the update timestamp.

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a query window titled "SQLQuery1.sql - 638...Administrator (53))" containing the following T-SQL code:

```
CREATE TRIGGER trg_AuditProductUpdates
ON Product
FOR UPDATE
AS
BEGIN
    DECLARE @product_id INT, @old_name VARCHAR(255), @old_category VARCHAR(255), @old_price DECIMAL(10,2), @old_stock INT;

    -- Get the old values of the updated product
    SELECT
        @product_id = product_id,
        @old_name = name,
        @old_category = category,
        @old_price = price,
        @old_stock = stock
    FROM deleted; -- The "deleted" table contains the old values of the updated record

    -- Insert into the AuditTrail table
    INSERT INTO AuditTrail (product_id, old_name, old_category, old_price, old_stock, operation)
    VALUES (@product_id, @old_name, @old_category, @old_price, @old_stock, 'UPDATE');
END;
```

The bottom pane shows the Messages tab with the following output:

```
Commands completed successfully.

Completion time: 2025-02-27T10:16:58.6303273+05:30
```

Tasks for test:

The image shows two screenshots of the SQL Server Enterprise Manager interface. The top screenshot displays a SQL query window with the following code:

```
-- Test the CalculateDiscountedPrice function
-- Update product details to trigger the audit trail
UPDATE Product SET price = 55000 WHERE product_id = 1;
-- Call the InsertProduct procedure
SELECT * FROM Product;
```

The bottom screenshot shows the same SQL query window with the following code:

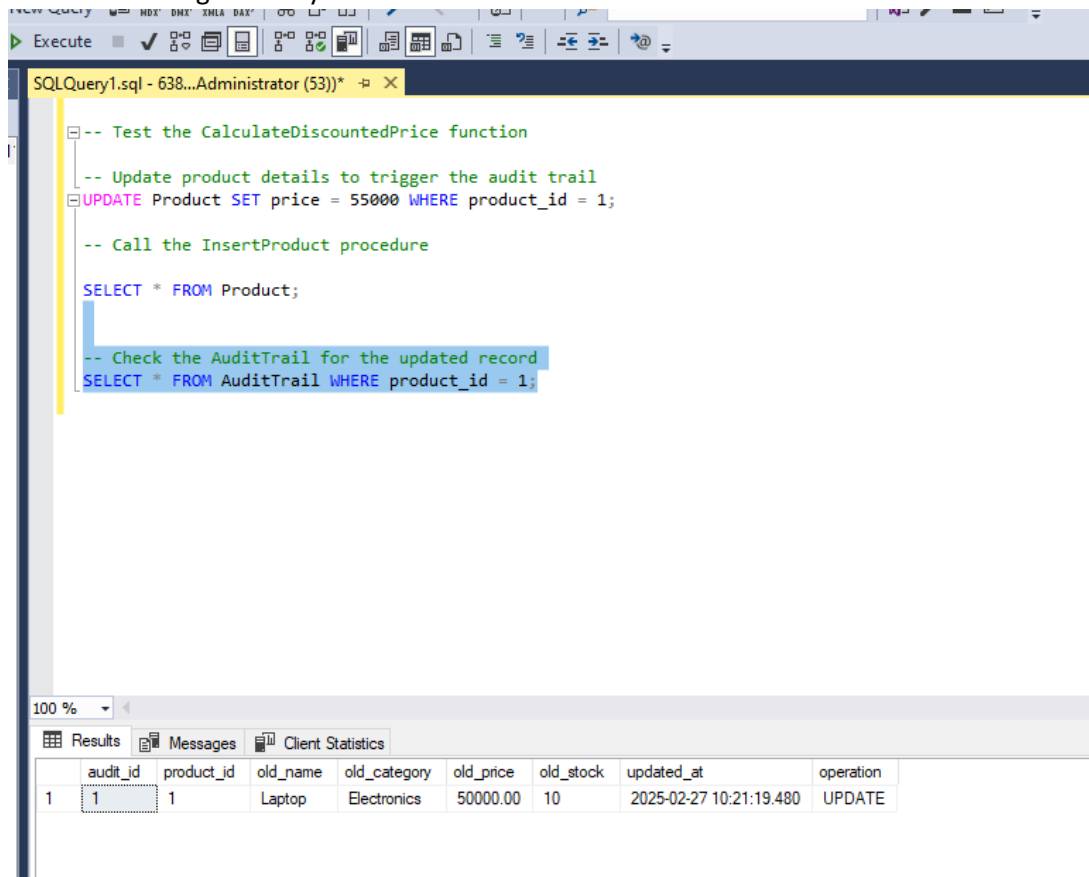
```
-- Test the CalculateDiscountedPrice function
SELECT dbo.CalculateDiscountedPrice(1000, 10); -- Price = 1000, Discount = 10%
SELECT * FROM Product;
```

Below the query window, the Results pane shows the output of the queries. The first screenshot shows the results of the `SELECT * FROM Product;` query, which returns a single row with the following data:

product_id	name	category	price	stock	
1	2	Laptop	Electronics	50000.00	10

The second screenshot shows the results of the `SELECT dbo.CalculateDiscountedPrice(1000, 10);` query, which returns a single row with the value 900.00:

(No column name)	
1	900.00



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query window titled "SQLQuery1.sql - 638...Administrator (53))" containing the following SQL script:

```
-- Test the CalculateDiscountedPrice function
-- Update product details to trigger the audit trail
UPDATE Product SET price = 55000 WHERE product_id = 1;
-- Call the InsertProduct procedure

SELECT * FROM Product;

-- Check the AuditTrail for the updated record
SELECT * FROM AuditTrail WHERE product_id = 1;
```

The bottom pane shows the "Results" tab with a grid displaying the output of the last query. The grid has 9 columns: audit_id, product_id, old_name, old_category, old_price, old_stock, updated_at, and operation. The first row shows the results for the audit trail query.

audit_id	product_id	old_name	old_category	old_price	old_stock	updated_at	operation
1	1	Laptop	Electronics	50000.00	10	2025-02-27 10:21:19.480	UPDATE